



Docker Documentation

A practical reference for Linux, Docker, AWS, and DevOps workflows

1. What is Docker?

Docker is a platform used to package and run applications inside containers.

A simple flow:

```
Application Code
  ↓
Dockerfile
  ↓
Docker Image
  ↓
Docker Container
  ↓
Application Running
```

Important terms

Term	Simple meaning
Docker	Tool/platform for containers
Image	Blueprint/template
Container	Running instance of an image
Dockerfile	Instructions for creating an image
Docker Hub	Registry for Docker images
Volume	Persistent storage
Network	Allows containers to communicate
Compose	Runs multiple containers together
Registry	Place where images are stored

2. Check Docker Installation

Check version:

```
docker --version
```

Example:

```
Docker version 28.x.x
```

Get detailed information:

```
docker info
```

Docker help:

```
docker --help
```

For a specific command:

```
docker run --help
```

3. Docker Images

An **image is a blueprint** from which containers are created.

List images:

```
docker images
```

or:

```
docker image ls
```

Download an image:

```
docker pull nginx
```

This downloads the `nginx` image from a container registry, normally Docker Hub when no registry is specified.

Remove an image:

```
docker rmi nginx
```

or:

```
docker image rm nginx
```

Inspect an image:

```
docker image inspect nginx
```

Search Docker Hub:

```
docker search nginx
```

4. Docker Containers

A **container is a running/created instance of an image**.

```
Image = Blueprint  
Container = Actual instance
```

Create and run a container:

```
docker run nginx
```

This:

```
nginx image  
  ↓  
creates container  
  ↓  
starts nginx
```

Run in background:

```
docker run -d nginx
```

`-d` means **detached/background mode**.

Give container a name:

```
docker run -d --name my-nginx nginx
```

Now the container is called:

```
my-nginx
```

5. View Containers

Running containers:

```
docker ps
```

All containers (includes stopped):

```
docker ps -a
```

Example:

CONTAINER ID	IMAGE	STATUS
abc123	nginx	Up 2 minutes

6. Start, Stop and Restart

Stop:

```
docker stop my-nginx
```

Start:

```
docker start my-nginx
```

Restart:

```
docker restart my-nginx
```

Kill immediately:

```
docker kill my-nginx
```

Difference:

```
stop → graceful stop  
kill → forcefully stop
```

7. Remove Containers

Remove a stopped container:

```
docker rm my-nginx
```

Force remove:

```
docker rm -f my-nginx
```

Remove all stopped containers:

```
docker container prune
```

⚠ Be careful with prune commands because they remove unused Docker objects.

8. Port Mapping

This is one of the **most important Docker concepts**.

Suppose your application runs inside the container on port `5000`, and you want to access it from your computer through port `5050`:

```
docker run -p 5050:5000 my-app
```

Meaning:

```
Your computer      Container
```

```
localhost:5050  —————>  :5000
```

General format:

```
docker run -p HOST_PORT:CONTAINER_PORT IMAGE
```

Example:

```
docker run -p 8080:80 nginx
```

Then open:

```
http://localhost:8080
```

9. Docker Logs

View container logs:

```
docker logs my-nginx
```

Follow logs continuously:

```
docker logs -f my-nginx
```

Useful when debugging applications.

Example:

```
docker logs -f backend
```

10. Execute Commands Inside Containers

Open a shell:

```
docker exec -it my-nginx bash
```

If the image doesn't have Bash:

```
docker exec -it my-nginx sh
```

Meaning:

```
Your terminal
  ↓
Docker container
  ↓
Shell inside container
```

Example:

```
ls
cd /usr/share/nginx/html
```

Exit:

```
exit
```

11. Dockerfile

A **Dockerfile** contains instructions for building an image.

Example Node.js application:

```
FROM node:22

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 5000

CMD ["npm", "start"]
```

Important Dockerfile instructions

Instruction	Meaning
FROM	Base image
WORKDIR	Working directory
COPY	Copy files
ADD	Copy/add files
RUN	Execute command while building
EXPOSE	Documents container port
CMD	Default command
ENTRYPOINT	Main executable/entrypoint
ENV	Set environment variable
ARG	Build-time variable

12. Build an Image

Suppose your project contains:

```
project/
├─ Dockerfile
├─ package.json
├─ package-lock.json
└─ server.js
```

Go into the project:

```
cd project
```

Build:

```
docker build -t my-node-app .
```

Meaning:

```
docker build
  ↓
Read Dockerfile
  ↓
Create image
  ↓
my-node-app
```

View it:

```
docker images
```

13. Image Tags

You can give an image a tag:

```
docker build -t my-node-app:1.0 .
```

Then:

```
docker images
```

You might see:

REPOSITORY	TAG
my-node-app	1.0

Run:

```
docker run my-node-app:1.0
```

14. `.dockerignore`

Similar to `.gitignore`.

Create `.dockerignore`, e.g.:

```
node_modules
.git
.env
*.log
```

This prevents unnecessary files from being sent to the Docker build context.

15. Environment Variables

Pass an environment variable:

```
docker run -e PORT=5000 my-node-app
```

Multiple variables:

```
docker run \
  -e PORT=5000 \
  -e NODE_ENV=production \
  my-node-app
```

Using an environment file:

```
docker run --env-file .env my-node-app
```

16. Volumes

Containers can be removed and recreated, so application data that needs to survive should generally be stored outside the container's writable layer.

A **Docker volume** provides persistent storage.

Create volume:

```
docker volume create my-data
```

List volumes:

```
docker volume ls
```

Use it:

```
docker run -v my-data:/data mongo
```

Meaning:

```
Docker volume
  ↓
/data inside container
```

Inspect:

```
docker volume inspect my-data
```

Remove:

```
docker volume rm my-data
```

17. Bind Mount

You can mount a folder from your computer into a container.

Example:

```
docker run -v ${pwd}:/app my-node-app
```

Meaning:

```
Your folder
  ↓
/app inside container
```

Bind mounts are commonly useful during development.

18. Docker Networks

Create a network:

```
docker network create my-network
```

List networks:

```
docker network ls
```

Run a container on it:

```
docker run -d \
  --name backend \
  --network my-network \
  my-backend
```

Another container:

```
docker run -d \
  --name mongodb \
  --network my-network \
  mongo
```

Containers on the same user-defined network can communicate using container/service names — the backend can connect to `mongodb:27017` rather than relying on a hard-coded container IP.

19. Docker Compose

If you have multiple services — React, Node.js, MongoDB — running them individually can become inconvenient.

Docker Compose lets you define the application stack in a YAML file.

Example:

```
services:

  backend:
    build: ./backend
    ports:
      - "5000:5000"

  mongodb:
    image: mongo
    volumes:
      - mongo-data:/data/db

volumes:
  mongo-data:
```

Start:

```
docker compose up
```

Start in background:

```
docker compose up -d
```

Stop:

```
docker compose down
```

View services:

```
docker compose ps
```

View logs:

```
docker compose logs
```

Follow logs:

```
docker compose logs -f
```

Rebuild:

```
docker compose up --build
```

20. Docker Hub / Registry

You can push your image to a registry.

Login:

```
docker login
```

Tag:

```
docker tag my-node-app username/my-node-app:latest
```

Push:

```
docker push username/my-node-app:latest
```

Another machine can then pull it:

```
docker pull username/my-node-app:latest
```

Then run it:

```
docker run username/my-node-app:latest
```

The flow:

```
Your Computer
  ↓
docker build
  ↓
Docker Image
  ↓
docker push
  ↓
Docker Hub
  ↓
docker pull
  ↓
Server / EC2
  ↓
docker run
  ↓
Container
```

21. Container Resource Usage

See resource usage:

```
docker stats
```

It shows:

```
CPU
Memory
Network
Block I/O
```

22. Inspect Containers

```
docker inspect my-nginx
```

This provides detailed information about the container, including:

- Network
- Mounts
- IP information
- Environment
- Configuration
- Container state

23. Docker System Information

Check Docker disk usage:

```
docker system df
```

Clean unused objects:

```
docker system prune
```

 `prune` can remove unused Docker resources. Read what it will remove before confirming.

24. Common Docker Commands Cheat Sheet

Images

```
docker images
docker pull nginx
docker build -t myapp .
docker rmi IMAGE
docker image inspect IMAGE
```

Containers

```
docker run IMAGE
docker run -d IMAGE
docker run --name myapp IMAGE
docker ps
docker ps -a
docker start CONTAINER
docker stop CONTAINER
docker restart CONTAINER
docker rm CONTAINER
```

Debugging

```
docker logs CONTAINER
docker logs -f CONTAINER
docker exec -it CONTAINER bash
docker inspect CONTAINER
docker stats
```

Ports

```
docker run -p 8080:80 nginx
```

Volumes

```
docker volume ls
docker volume create DATA
docker volume inspect DATA
docker volume rm DATA
```

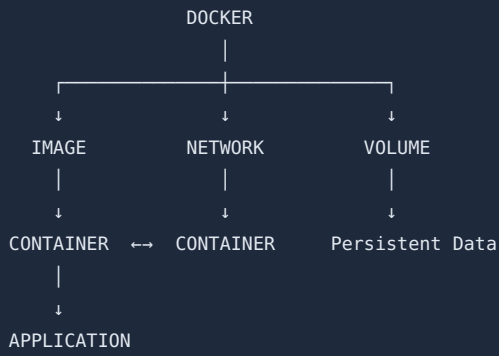
Networks

```
docker network ls
docker network create NETWORK
docker network inspect NETWORK
docker network rm NETWORK
```

Compose

```
docker compose up
docker compose up -d
docker compose down
docker compose ps
docker compose logs
docker compose up --build
```

☆ Core Docker Concepts



The core lifecycle:



Recommended learning order

1. Images & Containers
2. `docker run` , `ps` , `stop` , `rm`
3. Dockerfile & `docker build`
4. Port mapping
5. Logs & `exec`
6. Volumes
7. Networks
8. Docker Compose
9. Docker Hub / Registries
10. Docker + EC2
11. Docker + Kubernetes

Understanding these gives the foundation needed to move from **Docker** → **Kubernetes** → **AWS/DevOps**.